基於 SAM 3 語義置信度之人機協作影片標註系統

Efficiency-Driven Semi-Automated Data Engine:
Leveraging SAM 3 Presence Confidence for Human-in-the-Loop Video Annotation

施至遠 (Sonic)

國立陽明交通大學
輔助機器人研究群
Assistive Robotics Group, NYCU

2026

摘要

隨著無人水面載具（USV）技術的快速發展，海事環境感知系統對高品質訓練資料的需求日益增加。然而，傳統影片標註工作面臨標註效率低落、時序不一致、以及人工審核負擔過重等挑戰。本研究提出 STAMP（SAM Tracking Annotation with Minimal Processing）系統，一套基於 SAM 3 語義置信度的人機協作影片標註框架。

本系統的核心設計在於利用 SAM 3 的 Presence Score 作為信心導向審核的核心機制，將標註結果自動分類為高信心（≥ 0.80）、不確定（0.50–0.80）、低信心（< 0.50）三個等級。高信心標註自動接受，僅將不確定及低信心標註交由人類審核，從而將人類角色從「標註者」轉變為「審核者」。

本研究主要貢獻包括：

核心貢獻：

1. **首創 SAM 3 Presence Score 於人機協作標註之應用**：將視覺基礎模型的語義信心分數作為審核優先序指標，實現信心導向的人類審核分流
2. **標註效率量化基準**：建立總編輯操作次數（TEO）、每幀編輯率（EOR）、幀覆蓋率（FCR）、每物件點擊次數（CPO）、每幀秒數（SPF）等標準化指標
3. **完整的標註系統**：整合 Zero-Shot Discovery、Confidence-Guided Review、Interactive Refinement、Optimized GUI、Multi-format Export 等核心功能

場景依賴輔助模組：

- **Jitter Detection**：針對追蹤困難場景（遮擋、快速移動）的時序品質監控機制
- **Maritime ROI**：針對天空干擾場景的海平線偵測過濾機制（本研究以海事場景作為驗證平台）

實驗結果將量測：每幀編輯率（EOR）、每物件點擊次數（CPO）、每幀處理時間（SPF）等效率指標，以及標註品質（mIoU），與傳統手動標註方法（LabelMe）及半自動工具（CVAT + SAM）進行比較。

關鍵字：人機協作標註、信心導向審核、影片物件分割、Segment Anything Model

Contents

摘要 **1**

1 緒論 **10**

1.1 研究背景 10

1.1.1 標註效率挑戰 10

1.1.2 海事領域特殊挑戰 10

1.1.3 基礎模型的機會 10

1.2 問題陳述 11

1.3 研究目標 11

1.4 研究貢獻 11

1.4.1 核心學術貢獻 12

1.4.2 場景依賴輔助模組 12

1.4.3 實務貢獻 12

1.5 目標效能指標 12

1.6 論文架構 13

2 相關研究 **14**

2.1 分割基礎模型 14

2.1.1 Segment Anything Model (SAM) 14

2.1.2 SAM 2：影片擴展 14

2.1.3 SAM 3：Presence Score 創新 14

2.2 影片物件分割 15

2.2.1 記憶體方法 15

2.2.2 互動式 VOS 標註 15

2.3 標註效率優化 16

2.3.1 基於不確定性的方法 16

2.3.2 分割主動學習 16

2.3.3 與 STAMP 的關聯 17

2.4 半自動標註工具 17

2.4.1 傳統工具 17

2.4.2 SAM 系工具 17

2.4.3 領域特化工具 17
2.5 海事資料集 18
2.5.1 現有資料集 18
2.5.2 標註策略 18
2.6 小結 18

3 研究方法 20
3.1 系統概述 20
3.1.1 核心概念 20
3.1.2 系統架構 20
3.1.3 模組摘要 21
3.2 模組 1：Zero-Shot Discovery 22
3.2.1 可提示概念分割 22
3.2.2 Presence Score 擷取 22
3.3 模組 2：信心導向審核 22
3.3.1 三級分類 22
3.3.2 幀層級審核決策 23
3.3.3 效率指標體系 23
3.4 模組 3：時序追蹤與 Jitter 偵測 24
3.4.1 關鍵幀傳播 24
3.4.2 Jitter 偵測演算法 24
3.5 模組 4：互動式修正 25
3.5.1 點提示介面 25
3.5.2 每物件點擊次數（CPO）目標 26
3.6 模組 5：Maritime ROI 26
3.6.1 海平線偵測 26
3.7 模組 6：匯出模組 27
3.7.1 支援格式 27
3.7.2 自動資料集分割 27
3.8 模組 7：ActionLogger 27
3.8.1 設計理念 27
3.8.2 效率指標定義 27
3.8.3 記錄的操作類型 28
3.8.4 輸出格式 28
3.9 小結 28

4 系統實作 30
4.1 開發環境 30
4.1.1 硬體配置 30
4.1.2 軟體堆疊 30

4.1.3 Docker 容器化 31
4.1.4 Client-Server 架構 31
4.2 專案結構 31
4.3 核心模組實作 32
4.3.1 SAM3 引擎 32
4.3.2 Mock 模式開發 33
4.4 GUI 實作 33
4.4.1 主視窗架構 33
4.4.2 鍵盤快捷鍵 34
4.4.3 時間軸元件 34
4.5 VRAM 優化 35
4.5.1 記憶體挑戰 35
4.5.2 VRAM 估算公式 35
4.5.3 記憶體管理函式 35
4.5.4 風險等級警告系統 36
4.6 背景執行緒架構 36
4.7 小結 37

5 實驗設計 39
5.1 實驗目標 39
5.2 基準方法 39
5.2.1 基準工具 39
5.2.2 比較摘要 39
5.3 評估指標 40
5.3.1 效率指標 40
5.3.2 品質指標 41
5.4 資料集準備 41
5.4.1 測試影片 41
5.4.2 真實標註準備 41
5.5 實驗一：效率比較 42
5.5.1 目標 42
5.5.2 協定 42
5.5.3 預期結果 42
5.6 實驗二：品質比較 42
5.6.1 目標 42
5.6.2 協定 42
5.6.3 預期結果 43
5.7 實驗三：消融研究 43
5.7.1 目標 43

5.7.2 配置 43
5.7.3 預期結果 43
5.8 實驗四：Presence Score 有效性驗證 44
5.8.1 目標 44
5.8.2 比較方法 44
5.8.3 協定 44
5.8.4 預期結果 44
5.9 實驗五：Jitter 偵測有效性 44
5.9.1 目標 44
5.9.2 測試場景 45
5.9.3 指標 45
5.10 實驗六：使用者研究 45
5.10.1 目標 45
5.10.2 參與者 45
5.10.3 任務 45
5.10.4 測量 46
5.11 統計分析 46
5.11.1 顯著性檢定 46
5.11.2 報告 46
5.12 小結 46

6 結果與討論 47
6.1 實驗一：效率比較 47
6.1.1 結果 47
6.1.2 分析 47
6.2 實驗二：品質比較 48
6.2.1 結果 48
6.2.2 分析 48
6.3 實驗三：消融研究 48
6.3.1 結果 48
6.3.2 模組貢獻分析 48
6.4 實驗四：Presence Score 有效性驗證 49
6.4.1 結果 49
6.4.2 分析 49
6.5 實驗五：Jitter 偵測有效性 50
6.5.1 結果 50
6.5.2 分析 50
6.6 實驗六：使用者研究 51
6.6.1 結果 51

6.7 討論 51
6.7.1 關鍵發現 51
6.7.2 與文獻比較 52
6.7.3 限制 52
6.7.4 未來工作 52

7 結論 **53**
7.1 貢獻摘要 53
7.1.1 核心技術貢獻 53
7.1.2 場景依賴輔助模組 54
7.1.3 實務貢獻 54
7.2 研究問題回顧 54
7.3 限制 55
7.4 未來工作 55
7.4.1 短期擴展 55
7.4.2 研究方向 55
7.4.3 擴展性改善 56
7.5 結語 56

A 系統安裝指南 **59**
A.1 系統需求 59
A.2 安裝步驟 59
A.3 X11 設定 60

B API 參考 **61**
B.1 SAM3Engine 61
B.2 ConfidenceAnalyzer 63
B.3 JitterDetector 63

C 設定參數 **65**
C.1 信心閾值 65
C.2 Jitter 偵測閾值 65
C.3 匯出設定 66

D VRAM 參考表 **67**
D.1 VRAM 估算公式 67
D.2 各 GPU 安全幀數限制 67
D.3 處理時間估算 67

List of Figures

3.1 STAMP 系統資料流程 21

List of Tables

1.1 目標效能指標 13

2.1 SAM 模型家族比較 15
2.2 影片物件分割方法比較 16
2.3 標註工具比較 18
2.4 文獻關鍵指標 19

3.1 STAMP 模組摘要 21
3.2 效率指標定義 28
3.3 STAMP 系統參數 28

4.1 硬體配置 30
4.2 軟體堆疊 30
4.3 GUI 鍵盤快捷鍵 34
4.4 各影片配置的 VRAM 需求 35

5.1 基準方法功能比較 40
5.2 測試影片資料集 41
5.3 預期效率比較結果 42
5.4 預期品質比較結果 43
5.5 消融研究配置 43
5.6 預期消融研究結果 43
5.7 預期 Presence Score 有效性驗證結果 44
5.8 Jitter 偵測測試場景 45
5.9 實驗設計摘要 46

6.1 效率比較結果 47
6.2 品質比較結果 48
6.3 消融研究結果 48
6.4 Presence Score 有效性驗證結果 49
6.5 各場景 Jitter 偵測效能 50
6.6 NASA-TLX 工作負荷分數（越低越好） 51
6.7 與文獻基準比較 52

C.1 信心閾值設定 65
C.2 Jitter 偵測設定 65
C.3 匯出設定 66

D.1 各 GPU 與解析度的最大安全幀數 67
D.2 約略處理時間（RTX 4090） 67

Chapter 1

緒論

1.1 研究背景

無人水面載具（Unmanned Surface Vehicle, USV）技術的快速進展，對海事環境感知系統產生了前所未有的高品質訓練資料需求。現代深度學習物件偵測與分割模型需要大量標註資料才能在多變的海事環境中達到穩健的效能。然而，海事影片資料的標註工作面臨多重挑戰，嚴重影響效率與品質。

1.1.1 標註效率挑戰

傳統影片標註流程存在以下根本性限制：

- **勞力密集**：手動多邊形標註每幀需要 60–120 秒，操作者需要精確描繪物件邊界，每個物件需要 20 次以上的點擊
- **逐幀標註**：傳統工具如 LabelMe 和 CVAT 缺乏智慧時序傳播功能，導致相同物件在連續幀中需要重複標註
- **擴展性限制**：大規模海事資料集（1000 幀以上）需要數週的連續標註工作

1.1.2 海事領域特殊挑戰

海事領域引入了通用標註工具難以處理的額外複雜性：

- **鏡面反射**：水面產生動態反射，干擾分割模型判斷
- **海平線干擾**：天空區域經常產生假陽性偵測
- **低對比目標**：遠距船舶在海洋背景中呈現小型、低對比區域
- **動態遮擋**：波浪和水花可能暫時遮擋目標物件

1.1.3 基礎模型的機會

基礎模型的出現，特別是 Meta 的 Segment Anything Model（SAM）系列，徹底改變了互動式分割的格局。SAM 2 透過串流記憶體架構和模型迴圈資料引擎，展示了

8.4 倍的標註加速。最近發布的 SAM 3 進一步引入了可提示概念分割（Promptable Concept Segmentation, PCS）技術，其關鍵創新為 **Presence Head** 機制，可輸出語義層級的信心分數，表示「該概念是否存在於場景中」。

這個 Presence Score 提供了建構智慧標註系統的獨特機會—讓系統能夠評估自身的不確定性，實現從徹底人工標註到選擇性人工驗證的典範轉移。

1.2 問題陳述

儘管互動式分割技術持續進步，現有工具仍未解決一個根本性的效率問題：**無論 AI 信心程度如何，人類都必須審核每一個標註**。這導致：

1. **高信心偵測的人力浪費**：操作者花費時間驗證 AI 已經確定的標註
2. **品質不一致**：審核數千幀造成的疲勞導致真正有問題的標註被忽略
3. **缺乏時序品質控制**：追蹤失敗直到下游模型訓練時才會發現資料品質問題

本論文探討的研究問題是：

> *如何利用 AI 信心分數來智慧化地優先安排人類注意力，在維持品質的同時最小化標註工作量？*

1.3 研究目標

本論文提出 STAMP（SAM Tracking Annotation with Minimal Processing），一套將人類角色從「標註者」轉變為「審核者」的海事影片標註系統。具體目標如下：

1. **信心導向審核**：開發利用 SAM 3 Presence Score 將標註自動分類為 HIGH/UNCERTAIN/LOW 的框架，僅將不確定案例路由給人類審核者
2. **時序品質控制**：設計 Jitter Detection 演算法，監控連續幀間的遮罩形狀變化，自動標記追蹤失敗以供人類審核
3. **高效互動修正**：實作基於點提示的遮罩修正功能，以最少點擊實現快速修正（目標：每物件 < 2 次點擊）
4. **海事領域適應**：整合海平線偵測以限制分割於海事相關區域，減少天空區域的假陽性
5. **可投產系統**：提供完整的 GUI 應用程式，支援多格式匯出（COCO、Labelme、HuggingFace），可立即部署於研究工作流程

1.4 研究貢獻

本論文的貢獻如下：

1.4.1 核心學術貢獻

1. **首創 SAM 3 Presence Score 於人機協作標註之應用**：我們證明 Presence Head 的語義信心分數可作為審核優先序的有效指標，以顯著降低的人類介入達到相當的品質

2. **標註效率量化基準**：我們建立總編輯操作次數（TEO）、編輯操作率（EOR）、幀覆蓋率（FCR）、每物件點擊次數（CPO）、每幀秒數（SPF）等標準化指標作為標註效率基準

1.4.2 場景依賴輔助模組

針對特定挑戰場景，本系統提供以下輔助功能：

1. **Jitter Detection 時序品質控制**：結合 IoU 監控（閾值 0.85）與面積變化偵測（閾值 15%）的演算法，用於偵測遮擋、快速移動等場景下的追蹤失敗。在 SAM 3 追蹤穩定的一般場景下，此模組可選擇性啟用

2. **Maritime ROI 模組**：整合海平線偵測，用於過濾天空區域的假陽性。在目標清晰、無天空干擾的場景下，此模組可選擇性啟用

1.4.3 實務貢獻

1. **開源標註工具**：完整的 PyQt6 GUI 應用程式，配合 Docker 容器化以確保可重現部署

2. **多格式匯出**：直接匯出至 COCO JSON、Labelme JSON 和 HuggingFace Parquet 格式，並自動進行訓練/驗證/測試分割

3. **VRAM 優化**：提供經驗公式與警告系統，確保消費級 GPU 上的可靠處理

1.5 目標效能指標

表 1.1 總結了與傳統方法相比的目標效能。

Table 1.1: 目標效能指標

指標	定義	傳統方法	STAMP 目標
TEO	總編輯操作次數	與幀數成正比	最小化
EOR	每幀編輯率（TEO/總幀數）	≈ 1.0	< 0.05
FCR	幀覆蓋率（被編輯幀比例）	100%	$< 5\%$
CPO	每物件點擊次數	20+	< 2
SPF	每幀處理秒數	60–120 秒	< 5 秒
mIoU	標註品質	95%+	$> 90\%$

1.6 論文架構

本論文架構如下：

- **第二章：相關研究**回顧分割基礎模型、影片物件分割方法、標註效率優化策略、半自動標註工具及海事資料集
- **第三章：研究方法**呈現系統架構並詳述六大核心模組
- **第四章：系統實作**描述技術實作細節，包括 GPU 記憶體優化與 GUI 設計
- **第五章：實驗設計**定義實驗設置、基準方法及評估協定
- **第六章：結果與討論**分析實驗結果並與最新方法比較
- **第七章：結論**總結貢獻並討論未來工作

Chapter 2

相關研究

本章回顧與 STAMP 相關的五大研究方向：分割基礎模型、影片物件分割、標註效率優化（含主動學習）、半自動標註工具及海事資料集。

2.1 分割基礎模型

2.1.1 Segment Anything Model (SAM)

Segment Anything Model（SAM）[1] 引入了可提示分割的典範，透過點、框或文字提示實現零樣本遷移至新領域。其三階段資料引擎—從輔助手動到半自動再到全自動—建立了 SA-1B 資料集，包含超過 11 億個遮罩，涵蓋 1100 萬張影像。

2.1.2 SAM 2：影片擴展

SAM 2 [2] 透過串流記憶體機制將 SAM 架構擴展至影片，維護跨幀的物件表徵。主要創新包括：

- **記憶體注意力**：當前幀特徵與過去預測記憶庫之間的交叉注意力機制
- **模型迴圈資料引擎**：三階段標註工作流程，達成 **8.4 倍加速**（37.8 秒 → 4.5 秒/幀）
- **SA-V 資料集**：50.9K 影片，642.6K 遮罩軌跡，規模為先前 VOS 資料集的 53 倍

2.1.3 SAM 3：Presence Score 創新

SAM 3 引入可提示概念分割（PCS）與關鍵的 **Presence Head** 機制。不同於先前僅輸出分割遮罩的模型，Presence Head 提供語義層級的信心分數，表示「查詢的概念是否存在於場景中」。此分數可用於：

- 自動拒絕負面案例（概念不存在）

- 審核優先序的不確定性量化
- 透過文字提示進行多概念探索

表 2.1 比較 SAM 模型家族。

Table 2.1: SAM 模型家族比較

功能	**SAM**	**SAM 2**	**SAM 3**
影像分割	✓	✓	✓
影片分割	–	✓	✓
文字提示	–	–	✓
Presence Score	–	–	✓
記憶體機制	–	✓	✓

2.2 影片物件分割

影片物件分割（Video Object Segmentation, VOS）方法可分為半監督式（給定首幀遮罩）與互動式（人機迴圈）兩種方法。

2.2.1 記憶體方法

現代 VOS 方法以記憶體架構為主流，維護先前標註幀的表徵：

- **STCN** [3]：引入 VOS 記憶體匹配典範與高效注意力機制
- **XMem** [4]：提出受 Atkinson-Shiffrin 模型啟發的多儲存記憶體架構，於 DAVIS 2017 達成 86.2% J&F
- **Cutie** [5]：CVPR 2024 亮點論文，引入透過查詢變換器的物件層級記憶體讀取，於 DAVIS 2017 達成 **88.0% J&F**，同時速度為 DeAOT 的 3 倍
- **DeAOT** [6]：解耦關聯以實現高效多物件追蹤與階層傳播

2.2.2 互動式 VOS 標註

針對人機迴圈標註，多種方法整合使用者互動：

- **XMem++** [7]：擴展 XMem，加入基於注意力的**幀建議機制**，推薦最佳人類標註幀，達成 2 倍標註效率
- **Track Anything** [8]：結合 SAM 與 XMem 實現少點擊影片追蹤，無需額外訓練，於 DAVIS-2016 達成 92.0%

- **SAM-Track** [9]：整合 SAM、DeAOT 和 Grounding-DINO 支援多模態提示（點擊、筆劃、文字）

表 2.2 總結 VOS 方法效能。

Table 2.2: 影片物件分割方法比較

方法	會議	DAVIS 2017 J&F	速度	互動 GUI
STCN	NeurIPS 2021	85.4%	∼20 FPS	–
XMem	ECCV 2022	86.2%	∼20 FPS	展示 GUI
DeAOT	NeurIPS 2022	86.2%	22–53 FPS	部分
XMem++	ICCV 2023	–	即時	✓
Cutie	CVPR 2024	**88.0%**	∼30 FPS	✓
Track Anything	arXiv 2023	92.0%*	–	✓

*DAVIS-2016 結果

2.3 標註效率優化

主動學習策略性地選擇樣本進行標註，以顯著較少的標籤達到相當的模型效能。以下回顧相關方法。

2.3.1 基於不確定性的方法

- **Learning Loss for AL** [10]：CVPR 2019 經典論文，訓練損失預測模組來估計樣本難度，實現任務無關的主動學習
- **Evidential Deep Learning** [11]：ICLR 2023 論文，使用模型證據頭透過認識不確定性進行階層不確定性估計
- **PPAL** [12]：CVPR 2024 即插即用主動學習，具難度校準不確定性與類別條件匹配相似度

2.3.2 分割主動學習

- **MaskAL** [13]：基於 Mask R-CNN 的不確定性採樣，達成 **82% 資料減少**（僅用 17.9% 資料達到 93.9% 效能）
- **Multi-class Label Query** [14]：NeurIPS 2023 論文，透過多熱區域查詢達成 **96% 標註成本降低**—這是查詢設計的典範轉移，標註者指出所有存在的類別而非僅主要類別

2.3.3 與 STAMP 的關聯

STAMP 與傳統主動學習的關鍵差異在於：**我們不進行模型再訓練循環**。傳統主動學習透過「選資料 → 標註 → 再訓練」的迭代來改善模型；STAMP 則將信心用於**審核分流**——決定哪些幀需要人工審核。因此 STAMP 更精確地說是 **confidence-guided review system**，而非傳統主動學習框架。

具體差異：

1. 我們使用**模型自身的信心**（Presence Score）而非訓練獨立的不確定性估計器
2. 我們在**幀層級**運作，將整幀路由至人類審核，而非選擇樣本進行再訓練
3. 我們的目標是**減少人工審核負擔**，而非改善模型效能

2.4 半自動標註工具

2.4.1 傳統工具

- **LabelMe**：經典多邊形標註工具，完全手動
- **CVAT**：電腦視覺標註工具，內建追蹤器（TrackerMIL、CSRT），代表半自動標註的實務標準

2.4.2 SAM 系工具

- **Track Anything Annotate** [15]：arXiv 2025 生產工具，結合 SAM2 和 XMem++，支援匯出至 YOLO/COCO 格式
- **SegBuilder** [16]：WACV 2025 工具，透過 SAM 自動生成配合人類標記/修正，展示比 Label Studio 更快的效能

2.4.3 領域特化工具

- **FastCAT** [17]：醫療影片標註工具，透過兩階段工作流程達成 **20 倍專家時間減少**：

  1. *專家階段*：領域專家標註序列邊界（1.24 分鐘/影片）
  2. *非專家階段*：非專家在 AI 輔助下完成中間幀（2.93 秒/幀，相比 CVAT 的 5.79 秒）

  此工作流程為 STAMP 的信心驅動人類路由提供了模板。

表 2.3 比較標註工具。

Table 2.3: 標註工具比較

工具	類型	時間節省	關鍵功能
LabelMe	手動	基準	多邊形標註
CVAT	半自動	∼2 倍	內建追蹤
Track Anything	SAM 系	∼8 倍	少點擊傳播
FastCAT	領域特化	**20 倍**	兩階段工作流程
STAMP（本研究）	信心驅動	**5–10 倍**	Presence Score 路由

2.5 海事資料集

海事影片標註面臨獨特挑戰，包括鏡面反射、低對比目標和動態背景。

2.5.1 現有資料集

- **SeaShips** [18]：IEEE TMM 2018 基礎資料集，31,455 張影像，6 種船舶類型，用於海岸線監控
- **MarineInst20M** [19]：ECCV 2024 口頭報告，242 萬張影像，**1920 萬個遮罩**，6 種海事類別。關鍵洞察：**10% 人工標註遮罩訓練的模型生成了其餘 90%**，配合二元實例過濾進行品質控制
- **MVTD** [20]：arXiv 2025，首個海事影片物件追蹤基準，15 萬幀，具挑戰特定屬性

2.5.2 標註策略

MarineInst 的標註策略對 STAMP 特別具啟發性：

1. 10% 的遮罩由人工高精度標註
2. 這些標註訓練基礎模型
3. 模型生成其餘 90% 的標註
4. 二元實例過濾提供品質控制

這展示了人機迴圈系統如何擴展—**人類標註引導模型訓練，模型則大幅放大人類的努力**。

2.6 小結

表 2.4 綜合相關研究的關鍵指標，這些指標指導了 STAMP 的設計目標。

Table 2.4: 文獻關鍵指標

來源	指標	數值
SAM 2	標註加速	8.4 倍
SAM 2	每幀時間	4.5 秒
MaskAL	資料減少	82%
Multi-class Query	標註減少	96%
FastCAT	專家時間減少	20 倍
Cutie	DAVIS 2017 J&F	88.0%
XMem++	標註效率	2 倍

STAMP 綜合各方向的洞察：

- **基礎層**：SAM 3 互動式分割配合 Presence Score
- **審核分流**：信心導向審核實現選擇性人類路由
- **傳播骨幹**：時序追蹤配合 Jitter Detection 品質控制
- **工作流程設計**：FastCAT 啟發的自動與人類審核階段分離
- **領域適應**：Maritime ROI 實現海平線約束分割（場景依賴模組）

Chapter 3

研究方法

本章呈現 STAMP 系統架構並詳述各功能模組。設計理念以透過智慧信心路由將人類標註者從「標註者」轉變為「審核者」為中心。系統模組分為核心模組（任何場景皆適用）與場景依賴輔助模組。

3.1 系統概述

3.1.1 核心概念

STAMP 的根本創新可總結為：

> *「用 AI 信心分數來指揮人類，達成極致的標註效率。」*

傳統標註工作流程要求人類審核每一幀，無論 AI 信心程度如何。STAMP 引入信心驅動的分流系統：

1. SAM 3 自動偵測並分割所有符合文字提示的物件
2. 每個偵測結果獲得一個表示 AI 確定程度的 Presence Score
3. **高信心**（≥ 0.80）：自動接受，無需人類審核
4. **不確定**（0.50–0.80）：排入人類驗證佇列
5. **低信心**（< 0.50）：自動拒絕或標記審核
6. 人類審核者專注於不確定案例

3.1.2 系統架構

圖 3.1 說明完整的系統資料流程。

```
使用者輸入：文字提示（"ship, buoy"）
↓
模組 1：Maritime ROI（海平線偵測）
↓
模組 2：Zero-Shot Discovery（SAM3 + 文字提示）
↓
模組 3：Temporal Tracking（關鍵幀傳播）
↓
模組 4：Confidence Analysis + Jitter Detection
↓
↙            ↘
高信心              不確定/低信心
（自動接受）          （人類審核）
↓                ↓
模組 5：Interactive Refinement
↓                ↓
模組 6：Export（COCO/Labelme/HuggingFace）
```

Figure 3.1: STAMP 系統資料流程

3.1.3 模組摘要

表 3.1 總結系統模組，區分為核心模組與場景依賴輔助模組。

Table 3.1: STAMP 模組摘要

編號	模組	功能	來源	類型
1	Zero-Shot Discovery	文字提示物件偵測	SAM3 PCS	核心
2	Confidence-Driven AL	三級分類	自行開發	核心
3	Temporal Tracking	關鍵幀傳播	SAM3	核心
4	Interactive Refinement	點提示遮罩修正	SAM3 Point	核心
5	Optimized GUI	時間軸 + 快捷鍵	PyQt6	核心
6	Export Module	多格式輸出	自行開發	核心
7	ActionLogger	效率指標追蹤	自行開發	核心
8	Jitter Detection	追蹤失敗偵測	自行開發	場景依賴
9	Maritime ROI	海平線偵測	SegFormer	場景依賴

3.2 模組 1：Zero-Shot Discovery

3.2.1 可提示概念分割

不同於需要點擊特定物件的 SAM 2，SAM 3 的可提示概念分割（PCS）支援文字查詢：

```
# SAM3 影片會話
session_id = engine.start_video_session(video_path)

# 加入文字提示 - 探索所有符合的物件
result = engine.add_prompt(session_id, frame_idx=0, prompt="ship, buoy")

# 傳播至整部影片
all_results = engine.propagate(session_id)
```

Listing 3.1: Zero-Shot Discovery API 使用

3.2.2 Presence Score 擷取

對於每個偵測物件，SAM 3 的 Presence Head 輸出信心分數：

$$\text{Presence Score} = P(\text{概念存在}|\text{影像特徵}) \tag{3.1}$$

此分數範圍為 0 到 1，其中：

- 分數 ≈ 1.0：高度確定概念存在於偵測位置
- 分數 ≈ 0.5：對概念存在不確定
- 分數 ≈ 0.0：高度確定概念不存在

3.3 模組 2：信心導向審核

本模組是 STAMP 系統的核心。我們利用 SAM 3 的 Presence Score 將標註結果分類，決定哪些幀需要人工審核。這種設計繼承了主動學習的精神（以不確定性引導人力資源配置），但**不涉及模型再訓練**。因此更精確地說，本方法屬於 **confidence-guided review**（信心導向審核），而非傳統主動學習框架。

3.3.1 三級分類

ConfidenceAnalyzer 將每個偵測分類為三個類別：

$$\text{類別}(s) = \begin{cases} \text{HIGH} & \text{若} s \geq 0.80 \\ \text{UNCERTAIN} & \text{若} 0.50 \leq s < 0.80 \\ \text{LOW} & \text{若} s < 0.50 \end{cases} \tag{3.2}$$

其中 s 為 Presence Score。

閾值選擇說明：本研究採用 0.80 和 0.50 作為初始閾值，基於以下考量：(1) 0.80 為常見的高信心門檻；(2) 0.50 代表隨機猜測的機率邊界。這些閾值為啟發式選擇，未來工作將研究基於驗證資料的自適應閾值校準方法。

```python
class ConfidenceAnalyzer:
    def __init__(self,
                 high_threshold: float = 0.80,
                 low_threshold: float = 0.50):
        self.high_threshold = high_threshold
        self.low_threshold = low_threshold

    def categorize(self, score: float) -> ConfidenceCategory:
        if score >= self.high_threshold:
            return ConfidenceCategory.HIGH      # 綠色 - 自動接受
        elif score >= self.low_threshold:
            return ConfidenceCategory.UNCERTAIN # 黃色 - 審核
        else:
            return ConfidenceCategory.LOW       # 紅色 - 檢查/拒絕
```

Listing 3.2: 信心分類實作

3.3.2 幀層級審核決策

若**任何**偵測為 UNCERTAIN 或 LOW，則該幀需要人類審核：

$$\text{needs_review}(f) = \exists d \in f : \text{類別}(d.\text{score}) \neq \text{HIGH} \tag{3.3}$$

此保守方法確保沒有不確定標註逃脫審核。

3.3.3 效率指標體系

本系統採用以「編輯操作次數」為核心的指標體系：

1. **總編輯操作次數（TEO）**：

$$\text{TEO} = \sum_{i=1}^{N} \text{edit}_i \tag{3.4}$$

其中 edit_i 為第 i 次編輯操作（包含 Refine、Add、Delete、Merge、Swap 等）。

2. **編輯操作率（EOR）**：

$$\text{EOR} = \frac{\text{TEO}}{\text{總幀數}} \tag{3.5}$$

3. **幀覆蓋率（FCR）**：

$$\text{FCR} = \frac{|\{f : \text{使用者編輯}(f)\}|}{|\text{總幀數}|} \times 100\% \tag{3.6}$$

測量實際被編輯的唯一幀數佔總幀數的比例。

TEO 比傳統的「幀覆蓋率」更能反映真實工作量——例如同一幀被編輯 5 次，TEO 計為 5，而 FCR 僅計為 1 幀。

3.4 模組 3：時序追蹤與 Jitter 偵測

3.4.1 關鍵幀傳播

SAM 3 的影片預測器使用記憶體注意力將遮罩從標註的關鍵幀傳播至後續幀：

```
# 從具有初始遮罩的關鍵幀開始
session_id = engine.start_video_session(video_path)
engine.add_prompt(session_id, frame_idx=0, prompt="ship")

# 將遮罩傳播至整部影片
results = engine.propagate(session_id)
# 回傳 Dict[frame_idx, FrameResult]
```

Listing 3.3: 遮罩傳播

3.4.2 Jitter 偵測演算法

追蹤可能靜默失敗—模型繼續輸出不再對應原始物件的遮罩。我們提出 Jitter Detection 來識別此類失敗。

Jitter 類型

偵測四種 jitter 類型：

1. **low_iou**：遮罩形狀顯著變化（IoU < 0.85）
2. **area_spike**：遮罩面積變化 $> 15\%$
3. **appearance**：物件突然出現（前一幀不存在）
4. **disappearance**：物件突然消失

基於 IoU 的偵測

對於連續幀 $t-1$ 和 t：

$$\mathrm{IoU}(M_{t-1}, M_t) = \frac{|M_{t-1} \cap M_t|}{|M_{t-1} \cup M_t|} \tag{3.7}$$

若 IoU < 0.85，標記為 `low_iou` jitter。

面積變化偵測

$$\text{area_change} = \frac{\text{Area}(M_t) - \text{Area}(M_{t-1})}{\text{Area}(M_{t-1})} \tag{3.8}$$

若 $|\text{area_change}| > 0.15$，標記為 `area_spike`。

```
class JitterDetector:
    def __init__(self,
                 iou_threshold: float = 0.85,
                 area_change_threshold: float = 0.15,
                 min_mask_area: int = 100):
        self.iou_threshold = iou_threshold
        self.area_change_threshold = area_change_threshold
        self.min_mask_area = min_mask_area

    def detect_jitter(self, prev_mask, curr_mask):
        iou = self.compute_iou(prev_mask, curr_mask)
        area_change = self.compute_area_change(prev_mask, curr_mask)

        if iou < self.iou_threshold:
            return JitterEvent(type="low_iou", iou=iou)
        elif abs(area_change) > self.area_change_threshold:
            return JitterEvent(type="area_spike", area_change=area_change
    )
        return None
```

Listing 3.4: Jitter 偵測實作

3.5 模組 4：互動式修正

3.5.1 點提示介面

當需要人類審核時，使用者透過點提示修正遮罩：

- ***左鍵點擊***：正樣本點（「這是物件」）
- ***右鍵點擊***：負樣本點（「這不是物件」）

```
# 使用者在畫布上點擊
positive_points = [(x1, y1), (x2, y2)]  # 左鍵
negative_points = [(x3, y3)]            # 右鍵

# 轉換為 SAM3 格式
points = np.array(positive_points + negative_points)
labels = np.array([1]*len(positive_points) + [0]*len(negative_points))

# 修正遮罩
```

```
refined_mask = engine.refine_mask(
    image=current_frame,
    points=points,
    labels=labels,
    mask_input=current_mask  # 選配：先前遮罩
)
```

Listing 3.5: 互動式修正

3.5.2 每物件點擊次數（CPO）目標

傳統多邊形標註每物件需要 20 次以上點擊。使用點提示：

- 初始偵測：0 次點擊（透過文字提示自動完成）
- 小幅修正：1–2 次點擊
- 大幅修正：3–5 次點擊

目標：所有物件平均 **CPO** < 2。

3.6 模組 5：Maritime ROI

3.6.1 海平線偵測

Maritime ROI 模組將分割限制在海平線以下，消除天空區域的假陽性。

傳統方法

使用 Canny 邊緣和 Hough 線變換的邊緣式海平線偵測：

1. 應用 Canny 邊緣偵測
2. 使用 Hough 變換偵測直線
3. 過濾接近水平的線（角度 $< 15°$）
4. 選擇最強線作為海平線

SegFormer 方法

使用在 ADE20K 訓練的 SegFormer 進行語意分割：

1. 將影像分割為語意類別
2. 識別「天空」類別像素
3. 對天空/非天空邊界擬合直線
4. 回傳海平線參數

3.7 模組 6：匯出模組

3.7.1 支援格式

STAMP 以三種格式匯出標註：

1. **COCO JSON**：標準物件偵測格式，含訓練/驗證/測試分割
2. **Labelme JSON**：每幀 JSON + 影像，用於手動修正
3. **HuggingFace Parquet**：直接上傳至 HuggingFace Hub

3.7.2 自動資料集分割

```python
@dataclass
class ExportConfig:
    output_dir: Path
    base_name: str

    # 分割比例
    train_ratio: float = 0.8
    val_ratio: float = 0.1
    test_ratio: float = 0.1
    random_seed: int = 42

    # 選項
    include_rejected: bool = False
    include_hil_fields: bool = True  # 包含信心分數
```

Listing 3.6: 匯出設定

3.8 模組 7：ActionLogger

3.8.1 設計理念

ActionLogger 模組記錄使用者的所有編輯操作，以量化人類工作量。傳統的「幀覆蓋率」(FCR) 僅計算「有幾幀被編輯」，無法反映真實工作量——例如同一幀被編輯 5 次仍只算 1 幀。因此，本系統改採以「總編輯操作次數」(TEO) 為核心的指標體系。

3.8.2 效率指標定義

表 3.2 總結新的效率指標體系。

Table 3.2: 效率指標定義

指標	全名	定義	重要性
TEO	Total Edit Operations	總編輯操作次數	主要指標
EOR	Edit Operation Rate	TEO / 總幀數	主要指標
FCR	Frame Coverage Rate	被編輯幀數 / 總幀數	輔助指標
CPO	Clicks Per Object	總點擊數 / 物件數	輔助指標
SPF	Seconds Per Frame	總時間 / 總幀數	輔助指標

3.8.3 記錄的操作類型

ActionLogger 追蹤以下編輯操作：

- **APPLY_REFINE**：應用遮罩修正
- **PROPAGATE**：傳播修正至後續幀
- **ADD_OBJECT**：新增物件
- **DELETE_OBJECT**：刪除物件
- **MERGE_OBJECTS**：合併物件
- **SWAP_LABELS**：交換標籤

3.8.4 輸出格式

ActionLogger 輸出兩種檔案：

1. **JSONL**：逐行記錄每個操作的時間戳、類型、參數
2. **Summary JSON**：標註會話結束時的效率指標摘要

3.9 小結

表 3.3 總結所有模組的關鍵參數。

Table 3.3: STAMP 系統參數

模組	參數	數值
Confidence AL	高閾值	≥ 0.80
Confidence AL	低閾值	< 0.50
Jitter Detection	IoU 閾值	< 0.85
Jitter Detection	面積變化閾值	$> 15\%$
Jitter Detection	最小遮罩面積	100 像素
Export	訓練/驗證/測試分割	80/10/10

下一章將詳述這些模組的技術實作。

Chapter 4

系統實作

本章描述 STAMP 的技術實作，包括開發環境、系統架構、GUI 設計及 GPU 記憶體管理的關鍵優化。

4.1 開發環境

4.1.1 硬體配置

表 4.1 顯示開發硬體規格。

Table 4.1: 硬體配置

元件	規格
GPU	NVIDIA RTX 4090（24GB VRAM）
RAM	62GB 系統記憶體
儲存	NVMe SSD
作業系統	Ubuntu 24.04 LTS

4.1.2 軟體堆疊

Table 4.2: 軟體堆疊

元件	版本
Python	3.12
PyTorch	2.7.0+cu126
CUDA	13.0（驅動 580.95.05）
PyQt6	6.x（GUI 框架）
Docker	NVIDIA Container Toolkit

4.1.3 Docker 容器化

為確保可重現性，STAMP 使用 Docker 容器化：

```
# 建構映像
docker build -t stamp:latest -f docker/Dockerfile .

# 啟動伺服器（含 GPU）
docker compose up -d
```

Listing 4.1: Docker 部署

關鍵容器功能：

- NVIDIA Container Toolkit 提供 GPU 存取
- FastAPI 後端提供 RESTful API
- SAM3 作為 Git 子模組（`third_party/sam3/`）

4.1.4 Client-Server 架構

STAMP 採用 Client-Server 架構，將 GPU 推論與使用者介面分離：

- **Server**：Docker 容器內執行 FastAPI + SAM3 引擎，提供 GPU 推論服務
- **Client**：PyQt6 桌面應用程式，可透過 PyInstaller 打包為獨立執行檔

此架構的優點：

1. **資源集中**：多位標註者共用一台 GPU 伺服器
2. **部署簡化**：標註者只需安裝輕量 Client，無需 GPU 或 Docker
3. **維護便利**：模型更新僅需在伺服器端進行

4.2 專案結構

```
sam3_hil/
  docker/
      Dockerfile
      docker-compose.yml
  server/                          # FastAPI 後端
      main.py                      # 伺服器進入點
      task_manager.py              # 任務佇列管理
      routes/
          jobs.py                  # 任務 API
          video.py                 # 影片上傳 API
  src/
```

```
    core/
        video_loader.py          # 影片 I/O 含 LRU 快取
        sam3_engine.py           # SAM3 API 封裝
        confidence_analyzer.py   # 信心分類
        jitter_detector.py       # 時序品質控制
        exporter.py              # 多格式匯出
        maritime_roi.py          # 海平線偵測
    gui/
        main_window.py           # 主應用程式（本機模式）
        main_window_server.py    # 主應用程式（遠端模式）
    api_client.py                # HTTP 客戶端
models/
    Segformer/                   # 海平線偵測模型
third_party/
    sam3/                        # SAM3 子模組
```

4.3 核心模組實作

4.3.1 SAM3 引擎

SAM3Engine 類別提供統一 API，支援 GPU 推論與開發用 Mock 模式：

```python
class SAM3Engine:
    """SAM3 推論統一 API"""

    def __init__(self, mode: str = "auto"):
        """
        參數：
            mode: "gpu"、"mock" 或 "auto"
        """
        if mode == "auto":
            mode = "gpu" if torch.cuda.is_available() else "mock"

        if mode == "gpu":
            self._engine = SAM3GPUEngine()
        else:
            self._engine = SAM3MockEngine()

    # 影像推論
    def detect_image(self, image, prompt) -> FrameResult

    # 影片會話管理
    def start_video_session(self, video_path) -> str
    def add_prompt(self, session_id, frame_idx, prompt) -> FrameResult
```

```
    def propagate(self, session_id) -> Dict[int, FrameResult]
    def close_session(self, session_id)

    # 互動式修正
    def refine_mask(self, image, points, labels, mask_input) -> np.
   ndarray
    def propagate_mask(self, video_path, start_frame, mask,
                       points, labels, obj_id) -> Dict[int, np.ndarray]
```

Listing 4.2: SAM3Engine 架構

4.3.2 Mock 模式開發

Mock 模式支援無 GPU 的 GUI 開發：

```
class SAM3MockEngine(BaseSAM3Engine):
    """無 GPU 開發用 Mock 引擎"""

    def detect_image(self, image, prompt) -> FrameResult:
        # 生成隨機偵測與合成分數
        num_objects = random.randint(1, 5)
        detections = []

        for i in range(num_objects):
            mask = self._generate_random_mask(image.shape[:2])
            score = random.uniform(0.4, 0.95)  # 變化的信心
            detections.append(Detection(
                obj_id=i,
                mask=mask,
                box=self._mask_to_box(mask),
                score=score
            ))

        return FrameResult(frame_index=0, detections=detections)
```

Listing 4.3: 開發用 Mock 引擎

4.4 GUI 實作

4.4.1 主視窗架構

主視窗整合所有元件：

```
class MainWindow(QMainWindow):
    def __init__(self):
        # 核心元件
        self.video_loader = VideoLoader()
```

```python
        self.sam3_engine = None  # 延遲載入
        self.confidence_analyzer = ConfidenceAnalyzer()
        self.jitter_detector = JitterDetector()

        # 狀態管理
        self.results: Dict[int, FrameResult] = {}
        self.video_analysis: Optional[VideoAnalysis] = None
        self.object_status: Dict[int, str] = {}  # 接受/拒絕/待審

        # GUI 元件
        self._setup_ui()
        self._setup_shortcuts()
        self._connect_signals()
```

Listing 4.4: 主視窗結構

4.4.2 鍵盤快捷鍵

Table 4.3: GUI 鍵盤快捷鍵

按鍵	動作
空白鍵	播放/暫停影片
← / →	上一幀/下一幀
Home / End	第一幀/最後一幀
Tab	跳至下一個需審核幀
Enter	確認當前標註
Ctrl+O	開啟影片
Ctrl+S	儲存標註
Ctrl+Q	結束

4.4.3 時間軸元件

時間軸以顏色編碼顯示信心狀態：

- **綠色**：高信心，自動接受
- **黃色**：不確定，需審核
- **紅色**：低信心或偵測到 Jitter
- **藍色**：使用者編輯過的幀

4.5 VRAM 優化

4.5.1 記憶體挑戰

SAM3 影片處理為記憶體密集型。實證測試顯示：

- 600 幀 @ 1080p：約 22–23 GB VRAM
- 單次執行接近 RTX 4090 的 24GB 限制
- 連續執行會累積未釋放的記憶體

4.5.2 VRAM 估算公式

透過實證測試，我們推導出：

$$\text{VRAM (GB)} = 2.0 + 0.034 \times \text{幀數} \times \frac{\text{解析度}}{1080p} \tag{4.1}$$

表 4.4 顯示不同配置的 VRAM 需求。

Table 4.4: 各影片配置的 VRAM 需求

解析度	200 幀	400 幀	500 幀	600 幀	最大（24GB）
1920×1080	8.8 GB	15.6 GB	19.0 GB	22.4 GB	約 580 幀
1280×720	3.9 GB	6.9 GB	8.5 GB	10.0 GB	約 1300 幀
960×540	2.2 GB	3.8 GB	4.8 GB	5.6 GB	約 2900 幀

4.5.3 記憶體管理函式

```python
def clear_gpu_memory(force_full_clear: bool = False):
    """偵測前清理 GPU 記憶體"""
    gc.collect()

    if torch.cuda.is_available():
        torch.cuda.empty_cache()
        torch.cuda.synchronize()

        allocated = torch.cuda.memory_allocated() / 1024**3
        reserved = torch.cuda.memory_reserved() / 1024**3
        logger.info(f"GPU 已清理: {allocated:.2f}GB 配置, {reserved:.2f}GB 保留")

def estimate_vram_usage(frames: int, width: int, height: int) -> dict:
    """估算 VRAM 需求"""
    resolution_factor = (width * height) / (1920 * 1080)
    estimated_vram = 2.0 + 0.034 * frames * resolution_factor
```

```python

    gpu_info = get_gpu_memory_info()
    available = gpu_info.get("free", 24.0)

    if estimated_vram > available * 0.95:
        risk_level = "danger"
    elif estimated_vram > available * 0.80:
        risk_level = "warning"
    else:
        risk_level = "safe"

    return {
        "estimated_vram": estimated_vram,
        "available_vram": available,
        "risk_level": risk_level,
        "max_safe_frames": int((available * 0.80 - 2.0) /
                               (0.034 * resolution_factor))
    }
```

Listing 4.5: GPU 記憶體管理

4.5.4 風險等級警告系統

GUI 在處理前顯示 VRAM 警告：

```python
def _check_vram_before_detection(self, frames, width, height):
    estimate = estimate_vram_usage(frames, width, height)

    if estimate["risk_level"] == "danger":
        reply = QMessageBox.warning(
            self, "VRAM 警告",
            f"預估 VRAM: {estimate['estimated_vram']:.1f} GB\n"
            f"可用: {estimate['available_vram']:.1f} GB\n\n"
            f"OOM 風險高。建議最大: {estimate['max_safe_frames']} 幀。\n"
            "仍要繼續？",
            QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.
    No
        )
        return reply == QMessageBox.StandardButton.Yes

    return True
```

Listing 4.6: VRAM 警告對話框

4.6 背景執行緒架構

為維持推論期間的 GUI 響應性：

```python
class SAM3Worker(QThread):
    """SAM3 推論背景執行緒"""

    progress = pyqtSignal(int, str)  # (百分比, 訊息)
    finished = pyqtSignal(dict)       # 結果
    error = pyqtSignal(str)           # 錯誤訊息

    def run(self):
        try:
            self.progress.emit(5, "清理 GPU 記憶體...")
            clear_gpu_memory()

            self.progress.emit(10, "載入 SAM3 模型...")
            engine = SAM3Engine(mode=self.mode)

            self.progress.emit(30, "啟動影片會話...")
            session_id = engine.start_video_session(self.video_path)

            self.progress.emit(40, f"偵測中(提示: {self.prompt})...")
            engine.add_prompt(session_id, 0, self.prompt)

            self.progress.emit(50, "傳播遮罩...")
            results = engine.propagate(session_id)

            self.progress.emit(90, "關閉會話...")
            engine.close_session(session_id)
            engine.shutdown()

            self.progress.emit(100, "完成!")
            self.finished.emit({"results": results})

        except Exception as e:
            self.error.emit(str(e))
```

Listing 4.7: SAM3 背景執行緒

4.7 小結

關鍵實作亮點：

1. **Docker 容器化**確保可重現部署
2. **Mock 模式**支援無 GPU 的 GUI 開發
3. **VRAM 估算與警告**預防 OOM 錯誤
4. **背景執行緒架構**維持 GUI 響應性

5. **多格式匯出**支援多樣的下游工作流程

完整原始碼可於以下網址取得：https://github.com/ARG-NCTU/boats_dataset_processing

Chapter 5

實驗設計

本章呈現 STAMP 的實驗設計，包括基準方法、評估指標、資料集準備及實驗協定。

5.1 實驗目標

實驗旨在回答五個研究問題：

1. **RQ1（效率）**: STAMP 是否相較傳統方法減少標註時間？
2. **RQ2（品質）**: 基於信心的過濾是否維持標註品質？
3. **RQ3（模組貢獻）**: 各模組的貢獻為何？
4. **RQ4（信心指標有效性）**: Presence Score 是否為有效的審核優先序指標？
5. **RQ5（可用性）**: 使用者如何感知此系統？

5.2 基準方法

5.2.1 基準工具

1. **LabelMe**：完全手動多邊形標註，代表傳統基準。使用者在每一幀繪製物件邊界多邊形。
2. **CVAT + SAM**：電腦視覺標註工具，搭配內建 SAM（原版）進行半自動標註。使用者點擊物件，SAM 產生遮罩，代表目前業界標準的半自動標註工具。

5.2.2 比較摘要

表 5.1 比較基準功能。

Table 5.1: 基準方法功能比較

功能	LabelMe	CVAT + SAM	STAMP
SAM 輔助	–	✓（原版 SAM）	✓（SAM3）
文字提示	–	–	✓
自動傳播	–	部分	✓
信心路由	–	–	✓
Jitter 偵測	–	–	✓
點修正	–	✓	✓
Maritime ROI	–	–	✓

5.3 評估指標

5.3.1 效率指標

1. **總編輯操作次數（TEO）**：

$$\mathrm{TEO} = \sum_{i=1}^{N} \mathrm{edit}_i \tag{5.1}$$

其中 edit_i 為第 i 次編輯操作（包含 Refine、Propagate、Add、Delete、Merge、Swap）。

2. **每幀編輯率（EOR）**：

$$\mathrm{EOR} = \frac{\mathrm{TEO}}{\text{總幀數}} \tag{5.2}$$

3. **幀覆蓋率（FCR）**：

$$\mathrm{FCR} = \frac{\text{被編輯的唯一幀數}}{\text{總幀數}} \times 100\% \tag{5.3}$$

4. **每物件點擊次數（CPO）**：

$$\mathrm{CPO} = \frac{\text{總點擊次數}}{\text{唯一物件數}} \tag{5.4}$$

5. **每幀秒數（SPF）**：

$$\mathrm{SPF} = \frac{\text{總標註時間（秒）}}{\text{幀數}} \tag{5.5}$$

6. **標註加速**：

$$\text{加速} = \frac{\text{基準 SPF}}{\text{STAMP SPF}} \tag{5.6}$$

5.3.2 品質指標

1. **平均交集比（mIoU）**：

$$\text{mIoU} = \frac{1}{N}\sum_{i=1}^{N}\frac{|P_i \cap G_i|}{|P_i \cup G_i|} \tag{5.7}$$

其中 P_i 為預測遮罩，G_i 為真實標註。

2. **時序 IoU**：

$$\text{Temporal IoU} = \frac{1}{T-1}\sum_{t=1}^{T-1}\text{IoU}(M_t, M_{t+1}) \tag{5.8}$$

測量連續幀間的時序一致性。

3. **邊界 F1**：

$$\text{Boundary F1} = \frac{2 \cdot P_b \cdot R_b}{P_b + R_b} \tag{5.9}$$

遮罩邊界的精確率與召回率。

4. **物件召回率**：

$$\text{Recall} = \frac{\text{偵測到的物件}}{\text{真實標註物件總數}} \tag{5.10}$$

5.4 資料集準備

5.4.1 測試影片

表 5.2 描述測試影片集合。

Table 5.2: 測試影片資料集

編號	來源	幀數	解析度	挑戰
V1	台中港	500	1080p	多船舶、碼頭背景
V2	開放水域	400	1080p	水面反射、遠距船舶
V3	港口入口	300	1080p	船舶交會、遮擋
V4	夜間/低光	300	720p	低對比
V5	惡劣天氣	300	720p	波浪、霧氣

5.4.2 真實標註準備

真實標註透過 CVAT + 原版 SAM 準備，避免使用 SAM 3 以防止評估偏差：

1. 專家標註者使用 CVAT + 原版 SAM 輔助標註所有物件
2. 人工驗證並修正每個遮罩邊界
3. 由資深研究員進行最終品質審核

註：使用原版 SAM（非 SAM 3）確保真實標註與 STAMP 系統使用的模型獨立，避免評估偏差。

5.5 實驗一：效率比較

5.5.1 目標

驗證 STAMP 相較基準減少標註時間。

5.5.2 協定

1. **參與者**：3–5 位標註者（非專家）
2. **訓練**：每種工具 15 分鐘教學
3. **任務**：使用各工具標註測試影片 V1–V3
4. **平衡**：拉丁方設計控制順序效應
5. **測量**：時間、點擊次數、編輯幀數

5.5.3 預期結果

Table 5.3: 預期效率比較結果

方法	**SPF (秒)**	**CPO**	**EOR**	**加速**
LabelMe（手動）	待量測	待量測	–	1.0 倍
CVAT + SAM	待量測	待量測	待量測	待量測
STAMP（本研究）	待量測	待量測	待量測	待量測

5.6 實驗二：品質比較

5.6.1 目標

驗證基於信心的過濾維持標註品質。

5.6.2 協定

1. 使用實驗一的相同標註
2. 與專家真實標註比較
3. 計算 mIoU、Temporal IoU、Boundary F1

5.6.3 預期結果

Table 5.4: 預期品質比較結果

方法	mIoU (%)	Temporal IoU (%)	Boundary F1 (%)
LabelMe（手動）	待量測	–	待量測
CVAT + SAM	待量測	待量測	待量測
STAMP（本研究）	待量測	待量測	待量測

5.7 實驗三：消融研究

5.7.1 目標

量化各 STAMP 模組的貢獻。

5.7.2 配置

Table 5.5: 消融研究配置

配置	信心 AL	Jitter	ROI	修正
A1：完整系統	✓	✓	✓	✓
A2：移除信心 AL	–	✓	✓	✓
A3：移除 Jitter 偵測	✓	–	✓	✓
A4：移除 Maritime ROI	✓	✓	–	✓
A5：移除互動修正	✓	✓	✓	–

5.7.3 預期結果

Table 5.6: 預期消融研究結果

配置	EOR	mIoU (%)	SPF (秒)
A1：完整系統	待量測	待量測	待量測
A2：移除信心 AL	待量測	待量測	待量測
A3：移除 Jitter 偵測	待量測	待量測	待量測
A4：移除 Maritime ROI	待量測	待量測	待量測
A5：移除互動修正	待量測	待量測	待量測

5.8 實驗四：Presence Score 有效性驗證

5.8.1 目標

驗證 Presence Score 作為審核優先序指標的有效性。若 Presence Score 有效，則「低信心幀」應比「隨機選擇的幀」更需要人工修正。

5.8.2 比較方法

1. **隨機採樣**：隨機選擇幀進行審核
2. **均勻採樣**：均勻間隔的幀選擇
3. **不確定性採樣**：選擇平均信心最低的幀
4. **Presence Score（本研究）**: 具閾值的三級分類

5.8.3 協定

1. 固定審核預算為總幀數的 15%、10% 和 5%
2. 各方法在預算內選擇幀
3. 人類僅審核選定的幀
4. 比較審核後的最終 mIoU

5.8.4 預期結果

Table 5.7: 預期 Presence Score 有效性驗證結果

採樣方法	mIoU @ 15%	mIoU @ 10%	mIoU @ 5%
隨機	待量測	待量測	待量測
均勻	待量測	待量測	待量測
不確定性（熵）	待量測	待量測	待量測
Presence Score（本研究）	待量測	待量測	待量測

5.9 實驗五：Jitter 偵測有效性

5.9.1 目標

驗證 Jitter Detection 有效識別追蹤失敗。

5.9.2 測試場景

Table 5.8: Jitter 偵測測試場景

場景	描述	預期 Jitter 類型
快速移動	船舶快速移動	IoU 下降
遮擋	船舶被障礙物擋住	消失 → 出現
尺度變化	船舶接近/遠離	面積突變
相似物件	多艘相似船舶	IoU 下降
水面反射	表面干擾	面積突變

5.9.3 指標

$$\text{精確率} = \frac{\text{正確偵測的 Jitter}}{\text{總偵測 Jitter}} \tag{5.11}$$

$$\text{召回率} = \frac{\text{正確偵測的 Jitter}}{\text{實際追蹤失敗}} \tag{5.12}$$

5.10 實驗六：使用者研究

5.10.1 目標

評估使用者體驗與感知工作負荷。

5.10.2 參與者

- 3–5 位非專家標註者
- 1–2 位專家（品質驗證用）

5.10.3 任務

1. **任務 1**：使用 LabelMe 標註影片 A（100 幀）
2. **任務 2**：使用 CVAT 標註影片 B（100 幀）
3. **任務 3**：使用 STAMP 標註影片 C（100 幀）
4. **任務 4**：使用 STAMP 標註影片 D（300 幀）（擴展性測試）

影片隨機分配以避免順序效應。

5.10.4 測量

1. **客觀**：時間、點擊次數、錯誤、審核幀數
2. **主觀**：
    - NASA-TLX 工作負荷評估
    - 系統可用性量表（SUS）
    - 開放式回饋

5.11 統計分析

5.11.1 顯著性檢定

- 配對 t 檢定用於受試者內比較
- Wilcoxon 符號等級檢定用於非常態分布
- 顯著水準：$\alpha = 0.05$
- 效果量：Cohen's d

5.11.2 報告

結果將以下列格式報告：

- 平均值 ± 標準差
- 95% 信賴區間
- 統計顯著性（p 值）
- 效果量

5.12 小結

表 5.9 總結實驗設計。

Table 5.9: 實驗設計摘要

實驗	目標	基準	主要指標
實驗一	效率	LabelMe、CVAT + SAM	SPF、TEO、EOR
實驗二	品質	同實驗一	mIoU、Temporal IoU
實驗三	消融	模組變體	所有指標
實驗四	Presence Score 有效性	隨機、均勻、不確定性	mIoU vs 預算
實驗五	Jitter 偵測	無	精確率、召回率
實驗六	使用者研究	LabelMe、CVAT	NASA-TLX、SUS

Chapter 6

結果與討論

本章呈現實驗結果與分析。

6.1 實驗一：效率比較

6.1.1 結果

表 6.1 呈現各方法的效率比較。

Table 6.1: 效率比較結果

方法	**SPF（秒）↓**	**CPO ↓**	**TEO ↓**	**加速 ↑**
LabelMe（手動）	____ ± ____	____	____	1.0 倍
CVAT + SAM	____ ± ____	____	____	____ 倍
STAMP（本研究）	**_____ ± _____**	**_____**	**_____**	**_____ 倍**

6.1.2 分析

結果展示：

1. **SPF 降低**：STAMP 達成每幀約 ____ 秒，透過信心路由消除不必要的高信心幀審核
2. **CPO 改善**：文字提示實現零點擊初始偵測，整體 CPO 為 ____
3. **TEO 達標**：總編輯操作次數為 ____，EOR 為 ____%
4. **加速效果**：相較手動標註的 ____ 倍加速，對大規模資料集建立具有顯著實用價值

6.2 實驗二：品質比較

6.2.1 結果

Table 6.2: 品質比較結果

方法	mIoU（%）↑	Temporal IoU（%）↑	Boundary F1（%）↑
LabelMe（手動）	____ ± ____	–	____ ± ____
CVAT + SAM	____ ± ____	____ ± ____	____ ± ____
STAMP（本研究）	**_____ ± _____**	**_____ ± _____**	**_____ ± _____**

6.2.2 分析

1. **mIoU**：STAMP 達成 ____% mIoU。相較手動標註（____%），品質差異為 ____%

2. **時序一致性**：____% 的 Temporal IoU 顯示幀間連貫性，Jitter Detection 對品質維持有所貢獻

3. **品質-效率權衡**：結果驗證核心假設—以可接受的品質降低換取顯著效率提升是可行的

6.3 實驗三：消融研究

6.3.1 結果

Table 6.3: 消融研究結果

配置	EOR（%）	mIoU（%）	SPF（秒）	ΔmIoU
A1：完整系統	____	____	____	–
A2：移除信心 AL	____	____	____	____
A3：移除 Jitter 偵測	____	____	____	____
A4：移除 Maritime ROI	____	____	____	____
A5：移除互動修正	____	____	____	____

6.3.2 模組貢獻分析

1. **信心導向審核（A2）**：移除信心路由使 EOR 從 ____% 增加至 ____%，但維持品質。這展示信心過濾對效率的主要貢獻

2. **Jitter 偵測（A3）**：無 Jitter Detection 時，mIoU 下降 ____%，表示追蹤失敗未被偵測與修正。這驗證時序品質監控的重要性

3. **Maritime ROI（A4）**：移除 ROI 過濾導致 ____% mIoU 降低，主要因天空區域假陽性被納入

4. **互動修正（A5）**：最大的品質影響（____% mIoU 降低）發生於無修正功能時，展示人類修正對達成高品質仍不可或缺

6.4 實驗四：Presence Score 有效性驗證

6.4.1 結果

Table 6.4: Presence Score 有效性驗證結果

採樣方法	**mIoU @ 15%**	**mIoU @ 10%**	**mIoU @ 5%**
隨機	____%	____%	____%
均勻	____%	____%	____%
不確定性（熵）	____%	____%	____%
Presence Score（本研究）	**_____%**	**_____%**	**_____%**

6.4.2 分析

1. **Presence Score 有效性**：在所有預算水準，Presence Score 優於其他採樣策略，驗證其作為不確定性指標的使用

2. **預算敏感度**：即使在 5% 預算（僅 5% 的幀被審核），Presence Score 維持 ____% mIoU，相比隨機採樣的 ____%—____ 個百分點的優勢

3. **與不確定性採樣比較**：Presence Score 在 15% 預算時優於傳統不確定性採樣 ____ 個百分點，表示語義信心比基於熵的指標更有效地捕捉不確定性

6.5 實驗五：Jitter 偵測有效性

6.5.1 結果

Table 6.5: 各場景 Jitter 偵測效能

場景	真陽	假陽	精確率	召回率
快速移動	_____	_____	_____%	_____%
遮擋	_____	_____	_____%	_____%
尺度變化	_____	_____	_____%	_____%
相似物件	_____	_____	_____%	_____%
水面反射	_____	_____	_____%	_____%
整體	_____	_____	_____%	_____%

6.5.2 分析

1. **召回率**：_____% 的整體召回率表示 Jitter Detection 成功捕捉大多數追蹤失敗
2. **精確率**：_____% 精確率表示存在一些誤報，但這可接受—寧可多審核一幀也不要遺漏追蹤失敗
3. **場景特定效能**：
   - 快速移動和遮擋：高精確率和召回率
   - 水面反射：較低精確率，因自然面積變化被標記
   - 這暗示場景自適應閾值的潛力

6.6 實驗六：使用者研究

6.6.1 結果

NASA-TLX 工作負荷評估

Table 6.6: NASA-TLX 工作負荷分數（越低越好）

維度	**LabelMe**	**CVAT**	**STAMP**
心智負荷	_____	_____	_____
體力負荷	_____	_____	_____
時間壓力	_____	_____	_____
表現	_____	_____	_____
努力程度	_____	_____	_____
挫折感	_____	_____	_____
整體	_____	_____	_____

系統可用性量表（SUS）

STAMP SUS 分數：_____/**100**

關鍵正面回饋主題：

- 文字提示消除初始物件選擇
- 顏色編碼時間軸提供清晰指引
- Tab 跳至下一審核功能受讚賞
- 點擊修正直覺易用

改善空間：

- 長影片處理時間
- 信心閾值學習曲線
- 需要更好的進度回饋

6.7 討論

6.7.1 關鍵發現

1. **Presence Score 作為不確定性指標**：實驗驗證 SAM3 的 Presence Score 有效捕捉標註不確定性，實現高效的人機迴圈工作流程

2. **品質-效率平衡**：STAMP 達成有利的權衡：____ 倍加速僅伴隨相較手動標註 ____% 的 mIoU 降低

3. **模組協同**：各模組獨特貢獻：

   - 信心 AL：透過選擇性審核提升效率
   - Jitter 偵測：透過失敗偵測維持品質
   - Maritime ROI：透過約束進行領域適應
   - 互動修正：透過人類修正確保品質

6.7.2 與文獻比較

Table 6.7: 與文獻基準比較

來源	指標	報告值	STAMP
SAM2 [2]	加速	8.4 倍	_____ 倍
MaskAL [13]	資料減少	82%	_____%
FastCAT [17]	時間減少	20 倍	_____ 倍
Cutie [5]	DAVIS J&F	88.0%	_____%*

* 海事資料 Temporal IoU

6.7.3 限制

1. **VRAM 限制**：處理限於 24GB GPU 約 580 幀 @ 1080p
2. **領域特異性**：Maritime ROI 模組專屬海事環境
3. **文字提示依賴**：效能取決於提示品質
4. **閾值敏感度**：不同信心閾值可能產生不同結果

6.7.4 未來工作

1. **自適應閾值**：學習每影片/物件類型的最佳閾值
2. **多 GPU 支援**：長影片的分散處理
3. **主動提示改善**：基於偵測結果的自動提示優化
4. **遷移至其他領域**：非海事應用的評估

Chapter 7

結論

7.1 貢獻摘要

本論文提出 STAMP（SAM Tracking Annotation with Minimal Processing），一套利用 SAM 3 Presence Score 將人類標註者從「標註者」轉變為「審核者」的人機協作影片標註系統。核心設計在於使用基礎模型自身的信心評估來實現審核分流—自動接受高信心結果，同時將人類注意力導向不確定案例。本研究以海事場景作為驗證平台，但核心方法具跨領域適用性。

7.1.1 核心技術貢獻

1. **首創 SAM 3 Presence Score 於人機協作標註之應用**：我們證明 Presence Head 的語義信心分數可作為審核優先序的有效指標，使大部分幀得以自動接受而不影響品質。這是將基礎模型信心用於標註審核分流的實務應用。

2. **標註效率量化基準**：我們建立標準化的效率指標：

   - 總編輯操作次數（TEO）: 主要工作量指標
   - 每幀編輯率（EOR）: 主要效率指標
   - 幀覆蓋率（FCR）: 輔助指標
   - 每物件點擊次數（CPO）
   - 每幀秒數（SPF）

3. **完整標註系統**：我們實作並整合：

   - 透過文字提示的 Zero-Shot Discovery
   - 具三級分類的信心導向審核
   - 時序追蹤與傳播
   - 互動式點提示修正
   - 具快捷鍵的優化 PyQt6 GUI
   - 多格式匯出（COCO/Labelme/HuggingFace）

- ActionLogger 效率指標追蹤

7.1.2 場景依賴輔助模組

針對特定挑戰場景，本系統提供以下輔助功能：

1. **Jitter Detection 時序品質控制**：結合 IoU 監控（閾值 0.85）與面積變化偵測（閾值 15%）的演算法。此模組設計用於偵測遮擋、快速移動、相似物件混淆等場景下的追蹤失敗。實驗顯示，在 SAM 3 追蹤效果良好的一般場景下，此模組的貢獻有限，但在挑戰場景下可提供額外的品質保障。

2. **Maritime ROI 海平線偵測**：整合傳統邊緣偵測與 SegFormer 語意分割的海平線偵測模組。此模組設計用於過濾天空區域的假陽性偵測。在目標清晰、無天空干擾的場景下，此模組為選配功能。

7.1.3 實務貢獻

1. **可投產工具**：完整的 Docker 容器化應用程式，可立即部署於研究工作流程

2. **開源釋出**：程式碼可於以下網址取得：`https://github.com/ARG-NCTU/boats_dataset_processing`

3. **VRAM 優化**：經驗公式與警告系統，確保消費級 GPU 上的可靠處理

4. **文件**：完整的使用者指南與技術文件

7.2 研究問題回顧

本研究設計的實驗將回答五個研究問題：

1. **RQ1（效率）**: STAMP 相較手動標註能達成多少加速？信心路由是否能大幅減少標註時間？

2. **RQ2（品質）**: 在效率提升的同時，標註品質（mIoU）是否能維持在可接受水準（$> 90\%$）?

3. **RQ3（模組貢獻）**: 消融研究將量化各模組的貢獻：

    - 信心導向審核：預期為核心貢獻，大幅降低 EOR
    - 互動修正：預期為核心貢獻，確保標註品質
    - Jitter 偵測：場景依賴模組，預期在挑戰場景下提供品質保障
    - Maritime ROI：場景依賴模組，預期在天空干擾場景下減少假陽性

4. **RQ4（信心指標有效性）**：Presence Score 作為審核優先序指標，是否優於隨機採樣和傳統不確定性採樣？

5. **RQ5（可用性）**：使用者研究將評估 NASA-TLX 工作負荷與 SUS 可用性分數。

註：上述研究問題將於實驗完成後以實際數據回答。

7.3 限制

1. **硬體限制**：系統需要大量 GPU 資源（1080p 約 580 幀需 24GB VRAM），限制部分使用者的可及性。

2. **領域特異性**：Maritime ROI 模組專屬海事環境；適應其他領域需額外開發。

3. **提示敏感性**：系統效能取決於使用者提供的文字提示品質。

4. **閾值校準**：不同影片特性和物件類型的最佳信心閾值可能有所不同。

5. **單 GPU 架構**：SAM 3 目前不支援多 GPU 處理，限制超長影片的處理量。

7.4 未來工作

7.4.1 短期擴展

1. **批次處理**：實作多影片佇列處理與自動結果彙整

2. **時間軸增強**：加入縮放、選擇和標註預覽功能

3. **雲端部署**：雲端標註服務的 Docker Compose 配置

7.4.2 研究方向

1. **自適應閾值學習**：開發基於影片特性或使用者回饋自動校準信心閾值的方法

2. **主動提示改善**：研究基於初始偵測結果和使用者修正的自動提示優化

3. **多模態整合**：納入額外感測器模態（AIS、雷達、熱像）以增強偵測和自動標籤豐富化

4. **遷移至其他領域**：評估 STAMP 於非海事標註任務的適用性，如自動駕駛、監控和醫療影像

5. **聯邦學習**：支援從分散標註工作進行模型改善，同時保護資料隱私

7.4.3 擴展性改善

1. **分段處理流水線**：任意長度影片的自動分段與結果合併
2. **模型蒸餾**：研究維持 Presence Score 能力的較小模型，用於資源受限部署
3. **邊緣部署**：針對 USV 平台嵌入式系統優化，支援即時船上標註

7.5 結語

SAM 3 等基礎模型的出現，為資料標註的人機協作創造了前所未有的機會。本論文證明，透過利用模型內在的信心評估，我們可以將標註從徹底的人工標註轉變為智慧化監督—在維持品質的同時達成顯著的效率提升。

關鍵洞察是 **AI 信心賦能人類專注**。相較於審核每個標註，人類可以信任高信心結果，將精力集中於其判斷真正增加價值的案例。這個典範轉移—從「人類作為標註者」到「人類作為審核者」—代表我們處理大規模資料集建立方式的根本改變。

隨著基礎模型持續進步，我們預期人類介入率將進一步降低，趨近於真正自主的標註，僅對邊緣案例需要人類監督。STAMP 既提供當前標註需求的實用工具，也提供整合未來基礎模型進步的框架。

海事領域作為有效的測試平台，但這些原則具有廣泛適用性：信心驅動路由、時序品質控制和高效人機互動。我們期望本研究能為各電腦視覺應用帶來更高效的資料標註，使研究者能減少標註時間，投入更多精力於建構智慧系統。

Bibliography

[1] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, *et al.*, "Segment anything," *ICCV*, 2023.

[2] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, *et al.*, "Sam 2: Segment anything in images and videos," *ICLR*, 2025.

[3] H. K. Cheng, Y.-W. Tai, and C.-K. Tang, "Rethinking space-time networks with improved memory coverage for efficient video object segmentation," *NeurIPS*, 2021.

[4] H. K. Cheng and A. G. Schwing, "Xmem: Long-term video object segmentation with an atkinson-shiffrin memory model," *ECCV*, 2022.

[5] H. K. Cheng, S. W. Oh, B. Price, J.-Y. Lee, and A. G. Schwing, "Putting the object back into video object segmentation," *CVPR*, 2024.

[6] Z. Yang and Y. Yang, "Decoupling features in hierarchical propagation for video object segmentation," *NeurIPS*, 2022.

[7] M. Bekuzarov, J. Björklund, H. Farhadi, and A. Guirguis, "Xmem++: Production-level video segmentation from few annotated frames," *ICCV*, 2023.

[8] J. Yang, M. Gao, Z. Li, S. Gao, F. Wang, and F. Zheng, "Track anything: Segment anything meets videos," *arXiv preprint arXiv:2304.11968*, 2023.

[9] Y. Cheng, L. Li, Y. Xu, X. Li, Z. Yang, W. Wang, and Y. Yang, "Segment and track anything," *arXiv preprint arXiv:2305.06558*, 2023.

[10] D. Yoo and I. S. Kweon, "Learning loss for active learning," *CVPR*, 2019.

[11] J.-H. Park *et al.*, "Active learning for object detection with evidential deep learning and hierarchical uncertainty aggregation," *ICLR*, 2023.

[12] C. Yang, L. Huang, and E. J. Crowley, "Plug and play active learning for object detection," *CVPR*, 2024.

[13] P. M. Blok, E. J. van Henten, F. K. van Evert, *et al.*, "Active learning for instance segmentation using mask r-cnn," *WACV*, 2022.

[14] S. Hwang, S. Lee, H. Kim, J. Kim, *et al.*, "Active learning for semantic segmentation with multi-class label query," *NeurIPS*, 2023.

[15] D. Ivanov and A. Klimov, "Track anything annotate: Automatic video annotation for object detection and segmentation," *arXiv preprint*, 2025.

[16] M. A. Reza *et al.*, "Segbuilder: Segment-level annotation for interactive image segmentation," *WACV*, 2025.

[17] A. Krenzer, S. Heil, M. Makowski, A. Hekalo, C. Hoog Antink, F. Puppe, *et al.*, "Fastcat: Fast semi-automatic annotation of videos for medical analysis," *BME Online*, 2022.

[18] Z. Shao, W. Wu, Z. Wang, W. Du, and C. Li, "Seaships: A large-scale precisely annotated dataset for ship detection," *IEEE Transactions on Multimedia*, 2018.

[19] X. Zheng *et al.*, "Marineinst: A foundation model for marine vision with instance segmentation," *ECCV*, 2024.

[20] A. Bakht *et al.*, "Mvtd: Maritime video tracking dataset," *arXiv preprint*, 2025.

Appendix A

系統安裝指南

A.1 系統需求

- NVIDIA GPU，24GB 以上 VRAM（建議 RTX 4090）
- 安裝 NVIDIA Container Toolkit 的 Docker
- 具有 SAM 3 存取權限的 HuggingFace 帳號

A.2 安裝步驟

```
# 1. 連同子模組一起複製儲存庫
git clone --recursive git@github.com:ARG-NCTU/boats_dataset_processing.
    git
cd boats_dataset_processing/sam3_hil

# 2. 設定 HuggingFace 權杖
export HF_TOKEN="hf_your_token_here"

# 3. 建構 Docker 映像
./docker/docker_run.sh build

# 4. 執行容器
./docker/docker_run.sh shell

# 5. 容器內 - 啟動 GUI
python /app/src/gui/main_window.py
```

Listing A.1: 安裝指令

A.3 X11 設定

Linux 上的 GUI 顯示設定：

```
# 在主機端
xhost +local:docker

# 驗證顯示
echo $DISPLAY  # 應顯示 :0 或 :1
```

Listing A.2: X11 設定

Appendix B

API 參考

B.1 SAM3Engine

```python
class SAM3Engine:
    """SAM3 推論統一 API"""

    def __init__(self, mode: str = "auto"):
        """
        初始化 SAM3 引擎。

        參數：
            mode: "gpu"、"mock" 或 "auto"
        """

    def detect_image(
        self,
        image: np.ndarray,
        prompt: str
    ) -> FrameResult:
        """
        在單張影像上偵測物件。

        參數：
            image: BGR numpy 陣列 (H, W, 3)
            prompt: 文字提示（如 "ship, buoy"）

        回傳：
            包含偵測結果的 FrameResult
        """

    def start_video_session(self, video_path: str) -> str:
        """
        啟動影片處理會話。

```

```
        回傳：
            會話 ID 字串
        """

    def add_prompt(
        self,
        session_id: str,
        frame_index: int,
        prompt: str
    ) -> FrameResult:
        """在指定幀加入文字提示。"""

    def propagate(self, session_id: str) -> Dict[int, FrameResult]:
        """
        將遮罩傳播至整部影片。

        回傳：
            frame_index 到 FrameResult 的字典映射
        """

    def refine_mask(
        self,
        image: np.ndarray,
        points: np.ndarray,
        labels: np.ndarray,
        mask_input: Optional[np.ndarray] = None
    ) -> np.ndarray:
        """
        使用點提示修正遮罩。

        參數：
            image: BGR numpy 陣列
            points: (N, 2) [x, y] 座標陣列
            labels: (N,) 陣列，1（正）或 0（負）
            mask_input: 選配的先前遮罩

        回傳：
            修正後遮罩 (H, W)
        """

    def close_session(self, session_id: str) -> None:
        """關閉會話並釋放資源。"""
```

Listing B.1: SAM3Engine API

B.2 ConfidenceAnalyzer

```python
class ConfidenceAnalyzer:
    """分析 SAM3 信心分數。"""

    def __init__(
        self,
        high_threshold: float = 0.80,
        low_threshold: float = 0.50
    ):
        """
        初始化分析器。

        參數：
            high_threshold: 大於等於此值為 HIGH
            low_threshold: 小於此值為 LOW
        """

    def categorize(self, score: float) -> ConfidenceCategory:
        """分類單一分數。"""

    def analyze_frame(self, frame_result: FrameResult) -> FrameAnalysis:
        """分析幀中所有偵測。"""

    def analyze_video(
        self,
        results: Dict[int, FrameResult]
    ) -> VideoAnalysis:
        """
        分析整部影片。

        回傳：
            包含 FCR、類別計數等的 VideoAnalysis
        """

    def get_review_frames(
        self,
        results: Dict[int, FrameResult]
    ) -> List[int]:
        """取得需審核的幀列表。"""
```

Listing B.2: ConfidenceAnalyzer API

B.3 JitterDetector

```python
class JitterDetector:
```

```python
    """偵測時序追蹤失敗。"""

    def __init__(
        self,
        iou_threshold: float = 0.85,
        area_change_threshold: float = 0.15,
        min_mask_area: int = 100
    ):
        """
        初始化偵測器。

        參數：
            iou_threshold: 低於此值觸發 jitter
            area_change_threshold: 高於此值觸發 jitter
            min_mask_area: 考慮的最小像素
        """

    def compute_iou(
        self,
        mask1: np.ndarray,
        mask2: np.ndarray
    ) -> float:
        """計算兩遮罩間的 IoU。"""

    def analyze_object(
        self,
        obj_id: int,
        masks_by_frame: Dict[int, np.ndarray]
    ) -> ObjectJitterAnalysis:
        """分析單一物件的 jitter。"""

    def analyze_video(
        self,
        sam3_results: Dict[int, FrameResult]
    ) -> VideoJitterAnalysis:
        """分析整部影片的 jitter。"""
```

Listing B.3: JitterDetector API

Appendix C

設定參數

C.1 信心閾值

Table C.1: 信心閾值設定

參數	預設	範圍	效果
high_threshold	0.80	0.70–0.95	越高 = 越嚴格自動接受
low_threshold	0.50	0.30–0.60	越低 = 越多自動拒絕

C.2 Jitter 偵測閾值

Table C.2: Jitter 偵測設定

參數	預設	範圍	效果
iou_threshold	0.85	0.75–0.95	越低 = 越敏感
area_change_threshold	0.15	0.10–0.25	越低 = 越敏感
min_mask_area	100	50–500	忽略小遮罩

C.3 匯出設定

Table C.3: 匯出設定

參數	預設	說明
`train_ratio`	0.80	訓練集比例
`val_ratio`	0.10	驗證集比例
`test_ratio`	0.10	測試集比例
`random_seed`	42	可重現性種子
`include_rejected`	False	包含拒絕物件
`include_hil_fields`	True	加入信心至 COCO

Appendix D

VRAM 參考表

D.1 VRAM 估算公式

$$\text{VRAM（GB）} = 2.0 + 0.034 \times \text{幀數} \times \frac{\text{寬度} \times \text{高度}}{1920 \times 1080} \tag{D.1}$$

D.2 各 GPU 安全幀數限制

Table D.1: 各 GPU 與解析度的最大安全幀數

GPU（VRAM）	**1080p**	**720p**	**540p**	**360p**
RTX 3080（10GB）	約 200	約 450	約 1000	約 2200
RTX 3090（24GB）	約 580	約 1300	約 2900	約 6500
RTX 4090（24GB）	約 580	約 1300	約 2900	約 6500
A100（40GB）	約 1000	約 2300	約 5000	約 11000
A100（80GB）	約 2100	約 4700	約 10500	約 23000

D.3 處理時間估算

Table D.2: 約略處理時間（RTX 4090）

幀數	**模型載入**	**偵測**	**傳播**
100	30 秒	10 秒	20 秒
300	30 秒	15 秒	45 秒
500	30 秒	20 秒	90 秒
600	30 秒	25 秒	120 秒